UNIVERSITÀ DI PISA

MASTER OF SCIENCE IN COMPUTER ENGINEERING

Large Scale and Multi Structured Databases

SkyGraph

Project Members:

Luca Granucci

Lorenzo Marranini

Paolo Walsh

ACADEMIC YEAR 2025/2026

Contents

Chapter 1

Introduction

SkyGraph is a web application designed to combine real-time flight tracking with the analysis of historical flight data. The platform allows users to browse through statistics for airlines, airports, and specific routes, while also visualizing flights occurring in real-time on an interactive map.

While the system offers basic tracking features for Guest users, its core purpose is to support aviation professionals through specialized tools:

- **Traffic Controllers** use SkyGraph to ensure safety and manage airspace efficiency. The application allows them to monitor delay statistics and respond to critical situations. For example, in case of an emergency or an airport closure, controllers can quickly identify suitable landing sites or calculate alternative routes.

- **Airline Representatives** use the platform for strategic planning. By applying graph algorithms (such as Shortest Path and Centrality scores), they can evaluate the efficiency of existing connections and assess the potential for opening new commercial routes based on historical performance.

SkyGraph, therefore, provides solutions for both operational management and strategic decision- making.

The code has been uploaded to the following GitHub repository: https://github.com/lorenzo-marranini/SkyGraphProject

Chapter 2

Design

2.1 Actors

The application is designed to support four types of actors: *Guests*, *Traffic Controllers*, *Airline Representatives* and *Administrator*.

Guests

Guests are users who access the platform without registration. They interact with the system by utilizing the simpler functionalities, such as viewing currently flying planes and browse flights by some filters.

Traffic Controller

Traffic Controllers are registered users entrusted with the operational oversight of airport traffic and airspace safety. They represent the actors responsible for managing the daily flow of aircraft, responding to critical in-flight emergencies, and mitigating the impact of infrastructure disruptions.

Airline Representative

Airline Representatives are registered users who act on behalf of specific airline companies to manage strategic planning and operational analysis. Their role focuses on optimizing the airline's network efficiency and evaluating the creation of new routes.

Administrator

The administrator is a special user who can change the role of other users, create and delete accounts.

2.2 Requirements

2.2.1 Functional Requirements

Unregistered User

1. The system must allow unregistered users to search for flights based on filters such as Flight ID, Airport, Airline, and Date.
2. The system must provide a real-time map interface displaying the current position of active flights.
3. The system must allow unregistered users to access the Registration/Login page.
4. The system must allow unregistered users to create an account thus becoming a register user.

Registered User

1. The system must allow the Registered User to log in to the system.
2. The system must allow the Registered User to delete their own account.
3. The system must ensure that Registered Users retain all functionalities available to Unregistered Users.

Traffic Controller

Inherits all functionalities of a Registered User.

1. The system must allow the Traffic Controller to view statistics on the average delays of airlines.
2. The system must allow the Traffic Controller to view statistics on the total number of flights per airline.
3. The system must allow the Traffic Controller to view statistics on the total kilometers flown by airlines.
4. The system must allow the Traffic Controller to view statistics on the average flight distance flown by airlines.
5. The system must allow the Traffic Controller to view statistics on the number of flights operated by airlines on a selected route.
6. The system must allow the Traffic Controller to view statistics on the average delay of airlines on a selected route.

7. The system must allow the Traffic Controller to view information on airports where a selected flight could land in case of emergency.

8. The system must allow the Traffic Controller to view alternative routes in case an airport is unavailable.

Airline Representative

Inherits all functionalities of a Registered User.

1. The system must allow the Airline Representative to view possible flights by selecting a route and a number of stops.

2. The system must allow the Airline Representative to view statistics on the most frequent routes, including cancellations and diversions.

3. The system must allow the Airline Representative to view statistics on the betweenness centrality of airports.

4. The system must allow the Airline Representative to view statistics on the average delay of airports.

5. The system must allow the Airline Representative to view statistics on the connections of airports.

6. The system must allow the Airline Representative to view statistics on the most trafficked cities.

7. The system must allow the Airline Representative to view statistics on a selected city.

8. The system must allow the Airline Representative to view average delay statistics grouped by day of the week.

9. The system must allow the Airline Representative to view a comprehensive airline report containing the key performance metrics.

Administrator

1. The system must allow the Administrator to modify the role of an existing user.

2. The system must allow the Administrator to delete any user account.

3. The system must allow the Administrator to create new user accounts manually.

2.2.2 Non-Functional Requirements

Product Requirements

1. **Performance:** The system must ensure low latency for statistical queries and provide near real-time updates for the live flight map.
2. **Availability:** The system must be highly available to ensure continuous monitoring capabilities for Traffic Controllers.
3. **Scalability:** The system should be able to scale to accommodate increases in traffic and data load, for instance during peak holiday seasons or widespread weather emergencies, without degradation of response time.

Implementation Requirements

1. **Development:** The system must be developed using Object-Oriented Programming languages.
2. **API Standards:** The system must expose its functionalities via RESTful APIs, documented according to the OpenAPI Specification.
3. **Data Storage:** The system must utilize a NoSQL database structure to efficiently handle large volumes of unstructured flight data and complex aggregation queries.

Security Requirements

1. **Authentication & Authorization:** The system must implement a secure authentication protocol and enforce Role-Based Access Control to ensure users only access data pertinent to their role (e.g., Airline Representative and Traffic Controller).
2. **Data Protection:** The system must encrypt users' passwords using strong hashing algorithms.

2.3 UML Class Diagram

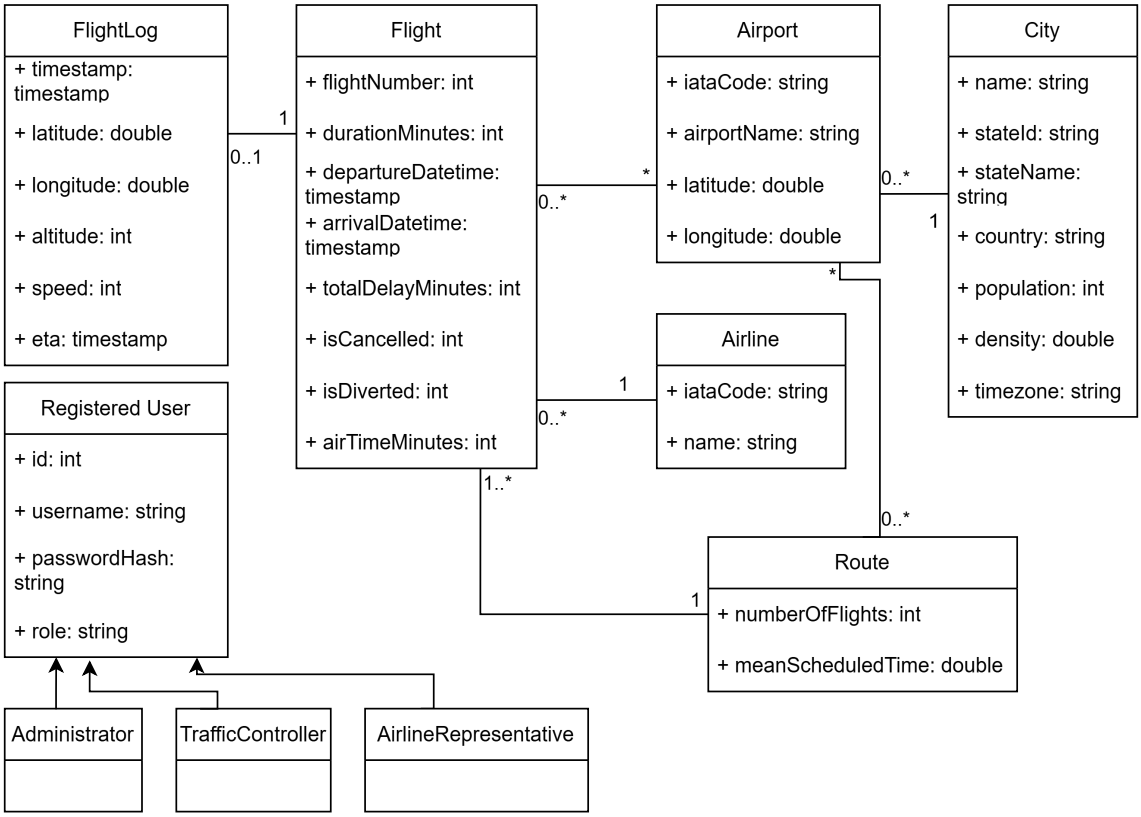

Figure 2.1: UML Diagram

2.4 Mock Ups

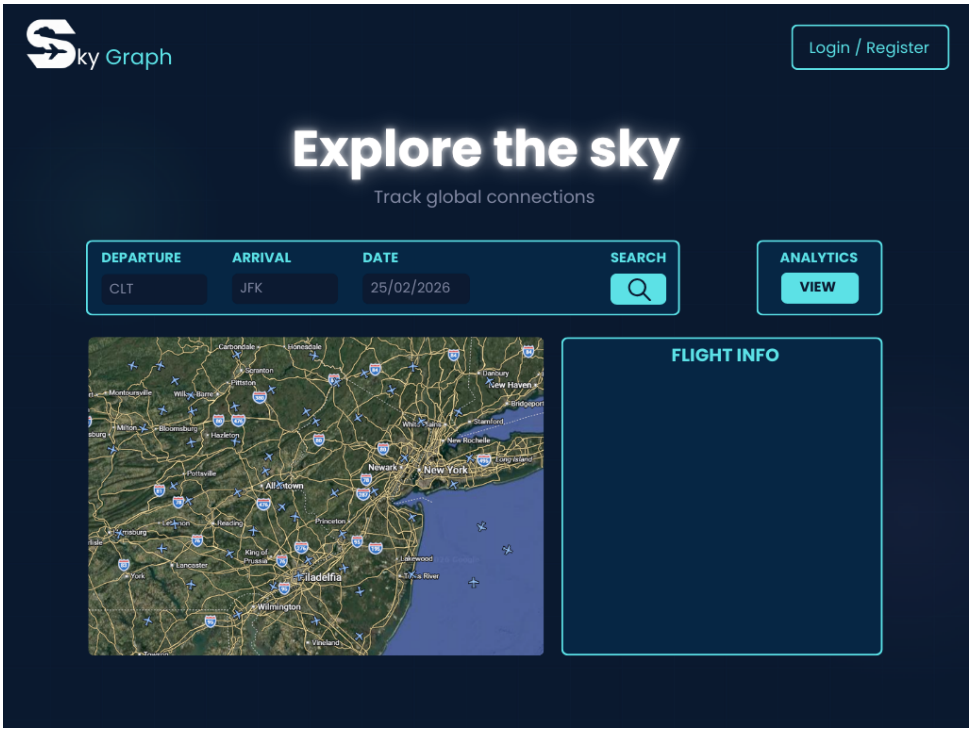

Figure 2.2: Home page

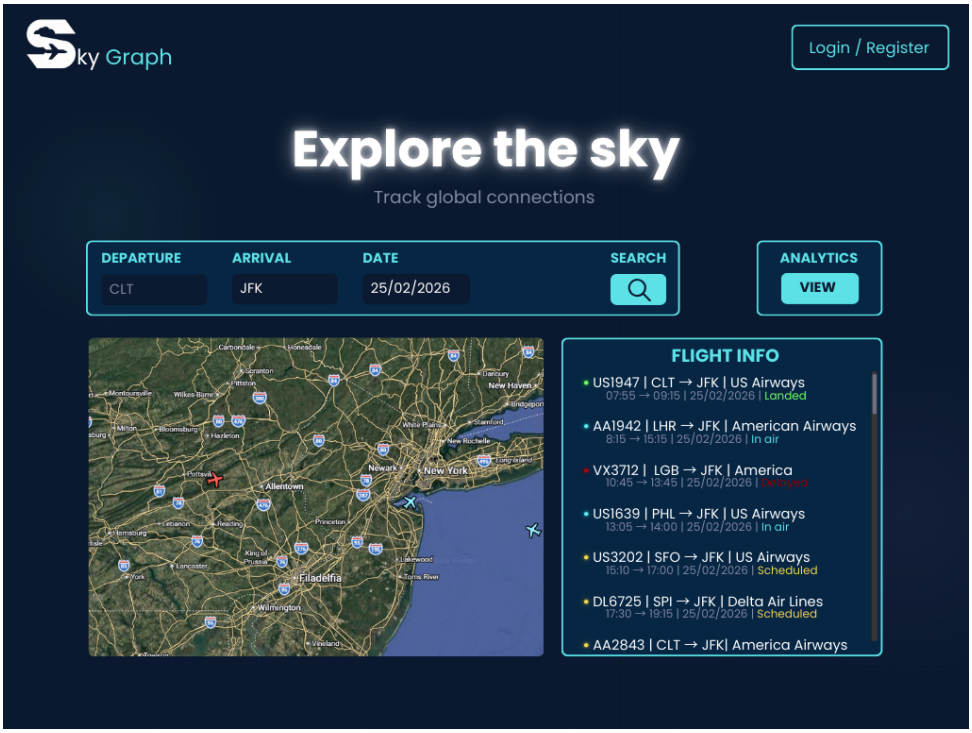

Figure 2.3: Live tracking and search

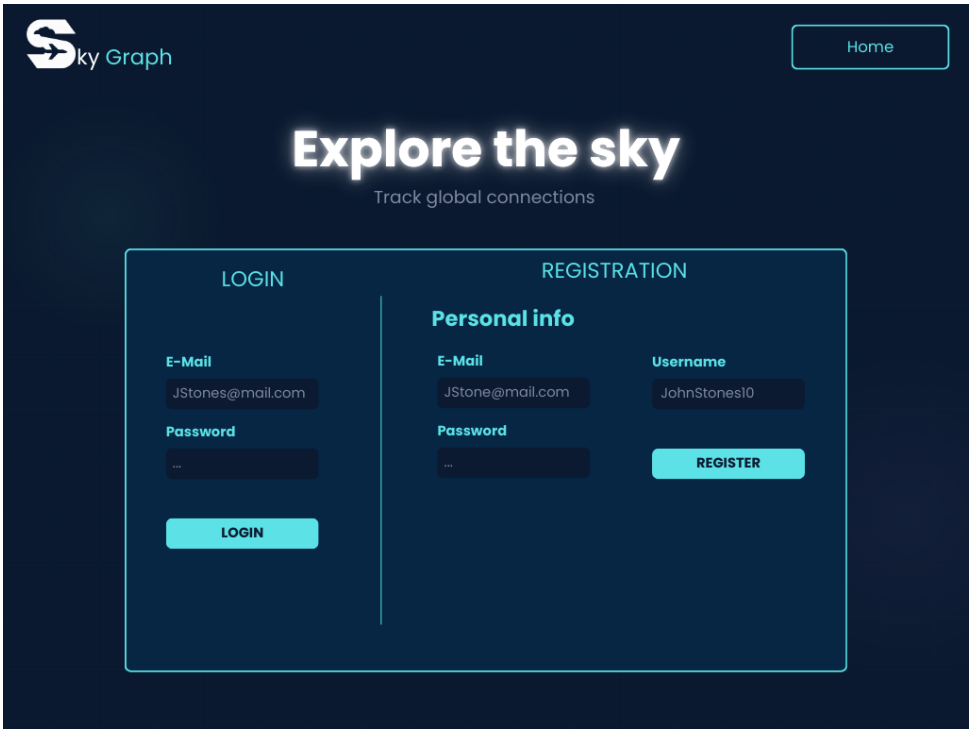

Figure 2.4: Login and registration

Figure 2.5: Traffic Controller Analytics 1/2

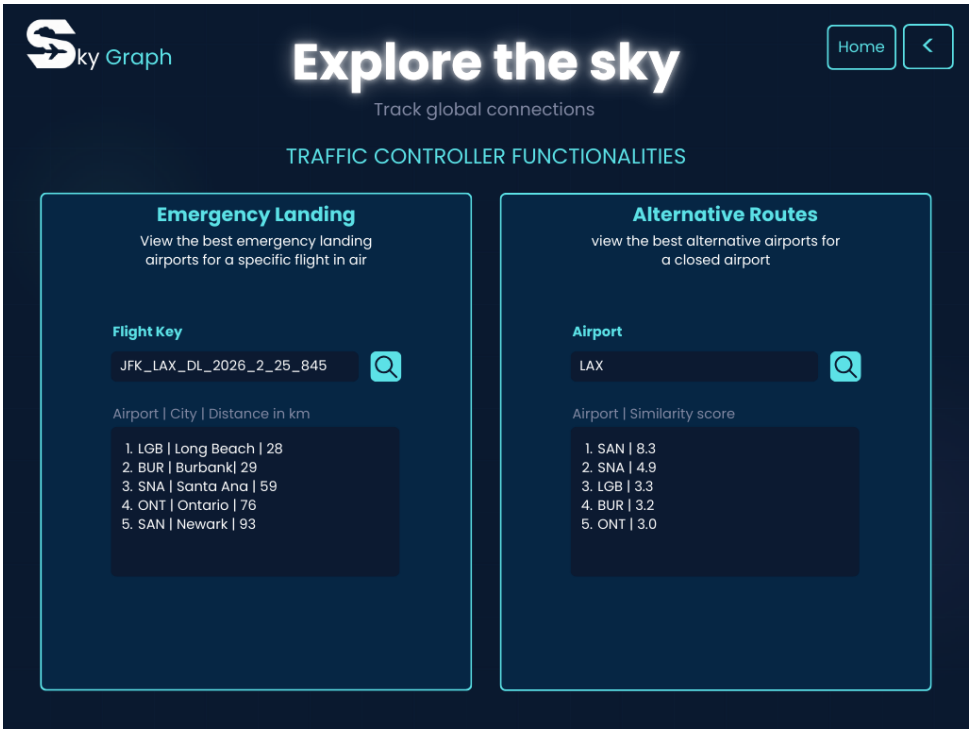

Figure 2.6: Traffic Controller Analytics 2/2

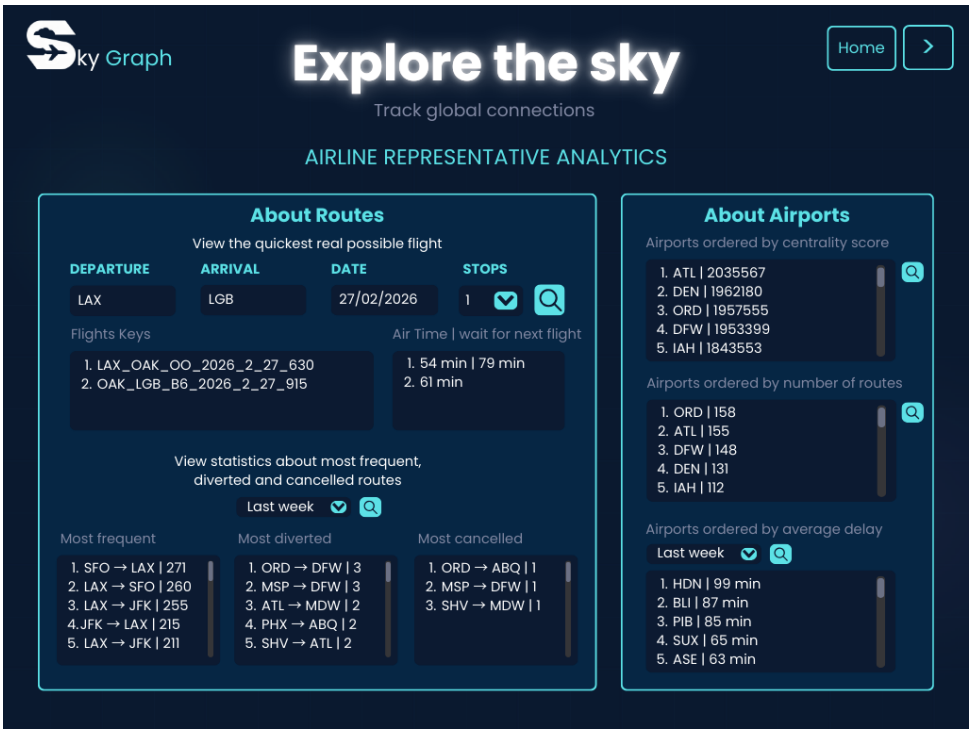

Figure 2.7: Airline Representative Analytics 1/2

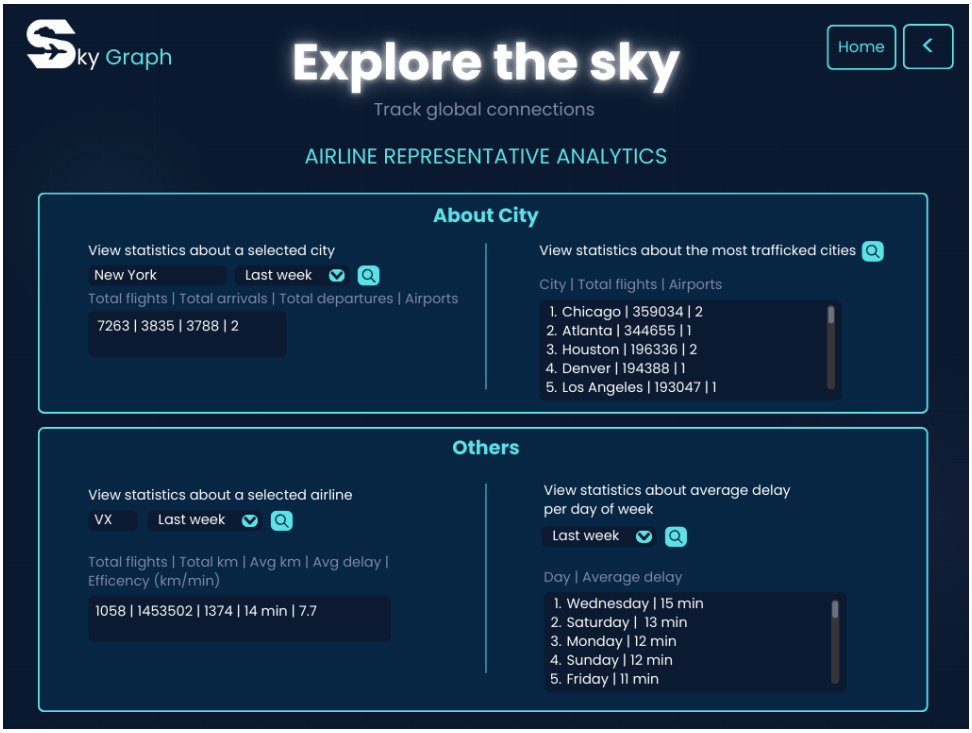

Figure 2.8: Airline Representative Analytics 2/2

2.5 Db Structure

2.5.1 Document Db

MongoDB was used to store platform data, creating the three following collections:

- Users
- Flights
- Airports

User Collection

The user collection stores information relating to registered users of the application. The main attribute of the document are:

- *username*: the name chosen by the user at the time of registration.
- *email*: the email the user used to register to the application.
- *password*: the (hashed) password that can be used to login.
- *role*: the role of the user, it can assume the value of, either `REGISTERED_USER`, `AIRLINE_REPRESENTATIVE`, and `TRAFFIC_CONTROLLER`.

```
{
    _id: 'a224aa61-2a18-474d-bde3-a3234cf770ea',
    username: 'pisa',
    email: 'pisa@gmail.com',
    password: '$2a$12$G/ogeQByJVrCstEYszmtCeyQd1txtJxm/
       NN9lHf1F0POamD898Nyq',
    role: 'ADMIN',
    _class: 'it.unipi.SkyGraph.model.UserMongo'
}
```

Flights Collection

The flight collection stores information regarding flights. Some important attributes of the collection are:

- *flight_info*: Contains core identification details, including the `airline_name` (e.g., "US Airways Inc."), `airline_code`, and the specific `flight_number`.
- *route*: Nested objects for `origin` and `destination` providing the IATA codes of the related airports and city names.
- *location*: uses GeoJSON-style "Point" types with" longitude and latitude `coordinates`, enabling spatial queries.
- *stats*: Key performance metrics including `tot_delay`, `air_time`, and status indicators for `cancelled` or `diverted` flights.
- *flight_log*: Live flight metrics such as `location`, `speed` and `altitude`.

```
{
  _id: ObjectId('69943126470400897dca43e6'),
  flight_info: {
    flight_key: 'JFK_AUS_AA_2026_2_25_726',
    airline: {
        iata: 'AA',
        name: 'American Airlines Inc.'
    },
    schedule: {
      departure_datetime: ISODate('2026-02-25T07
         :26:00.000Z'),
      arrival_datetime: ISODate('2026-02-25T10:40:00.000Z
         ')
    }
  },
  route: {
    origin: {
      iata: 'JFK',
```

```
      airport_name: 'John F. Kennedy International
         Airport (New York International Airport)',
      city: 'New York',
      state: 'NY',
      country: 'USA'
    },
    destination: {
      iata: 'AUS',
      airport_name: 'Austin-Bergstrom International
         Airport',
      city: 'Austin',
      state: 'TX',
      country: 'USA'
    },
    distance_km: 2554.24
  },
  stats: {
    tot_delay_minutes: null,
    is_cancelled: 0,
    is_diverted: 0,
    air_time_minutes: null
  },
  flight_log: {
    live_location: {
        type: 'Point',
        coordinates: [ -97.67045, 30.20174 ]
    },
    altitude: 0,
    speed: 2,
    timestamp: ISODate('2026-02-25T16:44:52.000Z'),
    eta: ISODate('2026-02-25T16:41:12.000Z'),
    _class: 'it.unipi.SkyGraph.model.
       FlightMongo$FlightLog'
  }
}
```

Airport

The Airport collection stores basic information about airports and its location in GeoJSON format.

```
{
  _id: 'ABE',
  name: 'Lehigh Valley International Airport',
  city: 'Allentown',
  state: 'PA',
  country: 'USA',
```

```
  location: { type: 'Point', coordinates: [ -75.4404,
     40.65236 ] }
}
```

2.5.2 Graph Db

Neo4j has been used to store the application data on routes, cities, and airports, as well as the connections between them. We employ two types of relationships: `LOCATED_IN`, which is a bidirectional relationship between a city and a related airport, and `ROUTE`, established between two airports, that signifies the existence of air traffic going from one airport to the other.

Airport Entity

<elementId>	4:1c0b6e8d-cce3-40c7-b48c-b19419cb50c5:299
<id>	299
iata_code	ACK
latitude	41.25305
longitude	-70.06018
name	Nantucket Memorial Airport

Table 2.1: Example of an Airport entity

The Airport entity of the graph stores position, iata code and name of the airports in the dataset.

City Entity

<elementId>	4:1c0b6e8d-cce3-40c7-b48c-b19419cb50c5:18
<id>	18
city_state	salt_lake_city_ut_usa
density	699
name	Salt Lake City
population	1098526
state_id	UT
timezone	America/Denver

Table 2.2: Example of a City entity

The city entity stores information on the city name, population, state, density, and timezone. Since city names are not unique, we identify a city with the city_state field, made by joining the name of the city with the state and country in which it is located.

Route Relationship

<elementId>	5:1c0b6e8d-cce3-40c7-b48c-b19419cb50c5:361
<id>	361
mean_scheduled_time	97.51515151515152
num_voli	1221

Table 2.3: Example of Route Relationship

The Route Relationship connects two Airports, and is present if at least one flight exists in the Mongo collection between two airports. The num_voli field allows us to quickly update the relationship when a flight is added in the flights collection, since we don't need to have other data to update mean_scheduled_time.

Graph Snapshot

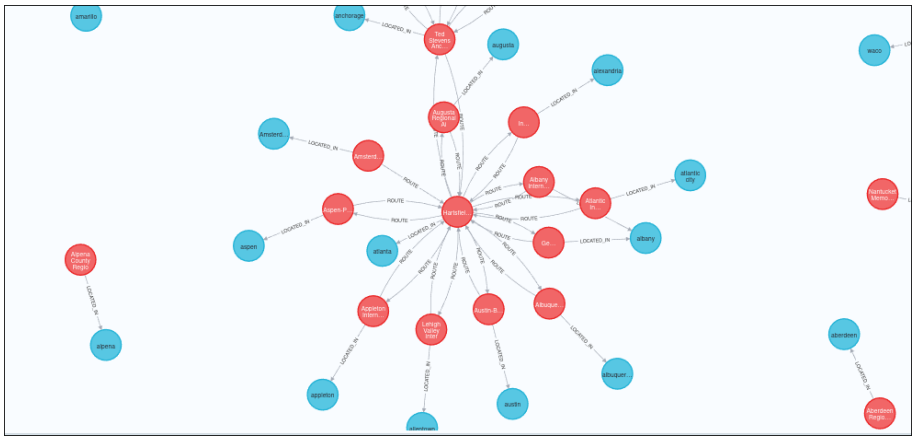

Figure 2.9: Sample screenshot of part of the graph

2.5.3 Consistency

Write operations are executed by always updating the Neo4j database first, and then the relative Mongodb collection, if present. If a Neo4j update fails, the operation is immediately aborted, ensuring that MongoDB remains entirely unmodified and the system stays consistent. Conversely, if a failure occurs during the MongoDB commit phase—after Neo4j has already been successfully updated, a critical warning is dispatched to a designated error log, and a manual rollback on the graph database will need to be executed. Intra-database consistency is ensured by the atomicity of single operations on MongoDb and Neo4j.

2.6 Replica Set and CAP Theorem Analysis

The MongoDB database of the application is deployed as a replica set consisting of three nodes across distinct Virtual Machines (IPs: `10.1.1.44`, `10.1.1.45`, and `10.1.1.46`). We implemented different policies to meet the specific requirements of our system.

2.6.1 CAP Theorem Implementation

In accordance with the CAP Theorem, we adjusted the read and write policies to balance performance and data integrity.
For the `flight_log` updates, which involve high-frequency uploading of data, we configured a `WriteConcern.W1` policy. By requiring acknowledgment from only the primary node, we prioritize availability and low latency, ensuring the tracking continues even during network congestion.
For sensitive operations such as flight status changes or analytics, we utilize a `majority` write concern. This ensures that data is replicated across at least two nodes before being confirmed, preventing data loss during failover.
To optimize the user experience we implemented a `nearest` read preference. This allows the application to fetch data from the node with the lowest network latency, favoring availability and responsiveness (AP) through an *eventual consistency* model.

While the physical infrastructure ensures partition tolerance (P) through the 3-node replica set, the software logic balances consistency (C) and availability (A) based on the data's criticality.

2.7 Sharding

While the current deployment relies on a single replica set, we have designed a horizontal scalability strategy to be implemented as the application grows[1]. Based on the estimated daily occurrence of different functionalities of our application reported in table 2.4, the ideal approach involves implementing a sharded cluster using a compound shard key: `{ "route.origin.iata": 1, "flight_info.flight_key": 1 }`.
This configuration is specifically designed to balance the 30,000 daily search queries with the 20,000 daily telemetry updates.

The first part of the key, the airport IATA code, serves as a prefix to ensure data locality. By grouping flights by their origin, the system allows the database to perform targeted queries for the majority of search operations, significantly reducing the overhead of data gathering operations across the cluster. However, relying solely

[1]The other collections, users and airports, have low enough volumes that sharding is not necessary

on the origin would lead to unbalanced shards, especially for major hubs like JFK or ATL which handle a disproportionate amount of traffic compared to smaller airports.

To prevent these "hotspots," the unique `flight_key` is included as the second part of the shard key to provide the necessary granularity for data distribution. Since each flight key is unique, it allows MongoDB to split large data chunks associated with a single busy airport. This ensures that the 20,000 daily log updates are distributed evenly across all three virtual machines.

Action	Daily Occurrences
Update flight log	20000
Auth Endpoints	
User login	10000
User registration	100
User Endpoints	
List users	100
Get user details	50
Assign user role	20
Delete own profile	10
Flight Endpoints	
Search flights	30000
Track Live flights	15000
Create flight	1000
Update flight	500
Delete flight	100
Quickest route	1500
Route rankings	500
Daily delay stats	500
Airport Endpoints	
List airports	1000
Get airport info	500
Add airport	>1
Update airport	>1
Delete airport	>1
Find alternatives	300
Airport rankings	500
Hub rankings	500
Connection rankings	500
Emergency search	500
City Endpoints	
List cities	1000
Add city	>1
City statistics	500
Delete city	>1
Traffic rankings	300
Airline Endpoints	
Airline report	1000
Airline rankings	500
Rankings by route	500

Table 2.4: Estimated Daily Occurrences per Action

Chapter 3

Dataset

The dataset used in this project aggregates information from two primary sources: the FlightRadar24 API, and the Kaggle datasets titled `2015 Flight Delays and Cancellations` and `United States Cities Database`. The Kaggle dataset served as the source for information regarding US flights, airports, whilst in order to gather data for the connected cities we used the second dataset. The FlightRadar24 data was utilized to capture real-time flight metrics.

3.1 Data Gathering Methodology

We constructed the initial flight collection using the Kaggle dataset. The dates of these flights were adjusted to create a temporal distribution where 70% of the flights occurred in the past and 30% are scheduled for the future.

To incorporate real-time tracking, we executed calls to the FlightRadar24 API, restricting the query scope to the geographical area of the United States and harvesting the resulting flight logs. We subsequently retrieved the specific flight summaries for these tracked flights, adapted the data structure to be compatible with the Kaggle dataset, and integrated them into the main flight collection. Finally, we added to the dataset the cities and airports for flights originated outside the US.

3.2 Volumes

The following are the volumes of the gathered data

- 5 Million Flights
- 6500 Flight Logs
- 311 Cites
- 342 Airports

Chapter 4

Implementation

The application is developed using Spring Boot, which facilitates the development through automatic configuration and embedded server deployment. For access control, the system integrates Spring Security with a JWT-based authentication strategy. This ensures secure and persistent user sessions without maintaining server-side state. The authorization logic is enforced via a filter chain:

Public Endpoints: Routes for user registration, authentication, and guest flight search are whitelisted and bypass the security filter.

Protected Endpoints: All other routes require a valid Bearer Token in the HTTP header, allowing the system to validate the user's role (Traffic Controller, Airline Representative or Administrator) before processing the request.

4.1 Model

We now present the models defined in our spring boot application that map directly to the database entities.

4.1.1 MongoDB Models

UserMongo

```
public class UserMongo {

    @Id
    private String id;

    @Indexed(unique = true)
    private String username;

    @Indexed(unique = true)
    private String email;

    private String password;
```

```
    private Role role;
}
```

UserPrincipal

```
public class UserPrincipal implements UserDetails {

    private final UserMongo user;

    public UserPrincipal(UserMongo user) {
        this.user = user;
    }

    @Override
    public Collection<? extends GrantedAuthority>
       getAuthorities() {
        return Collections.singletonList(new
           SimpleGrantedAuthority("ROLE_" + user.getRole
           ().name()));
    }

    @Override
    public String getPassword() {
        return user.getPassword();
    }

    @Override
    public String getUsername() {
        return user.getEmail();
    }

    @Override
    public boolean isAccountNonExpired() { return true; }

    @Override
    public boolean isAccountNonLocked() { return true; }

    @Override
    public boolean isCredentialsNonExpired() { return
       true; }

    @Override
    public boolean isEnabled() { return true; }
}
```

FlightMongo

```
public class FlightMongo {

    @Id
    private String id;

    @Field("flight_info")
    private FlightInfo flightInfo;

    private Route route;

    private Stats stats;

    @Field("flight_log")
    private FlightLog flightLog;

    @Data
    @NoArgsConstructor
    @AllArgsConstructor
    public static class FlightInfo {
        @Field("flight_key")
        private String flightKey;

        private Airline airline;

        private Schedule schedule;
    }

    @Data
    @NoArgsConstructor
    @AllArgsConstructor
    public static class Airline {
        private String iata;
        private String name;
    }

    @Data
    @NoArgsConstructor
    @AllArgsConstructor
    public static class Schedule {

        @Field("departure_datetime")
        private Instant departureDatetime;

        @Field("arrival_datetime")
        private Instant arrivalDatetime;
```

```
    }

    @Data
    @NoArgsConstructor
    @AllArgsConstructor
    public static class Route {
        private AirportDetails origin;

        private AirportDetails destination;

        @Field("distance_km")
        private Double distanceKm;
    }

    @Data
    @NoArgsConstructor
    @AllArgsConstructor
    public static class AirportDetails {
        private String iata;

        @Field("airport_name")
        private String airportName;

        private String city;
        private String state;
        private String country;
    }

    @Data
    @NoArgsConstructor
    @AllArgsConstructor
    public static class GeoLocation {
        private String type;
        private List<Double> coordinates;
    }

    @Data
    @NoArgsConstructor
    @AllArgsConstructor
    public static class Stats {
        @Field("tot_delay_minutes")
        private Integer totalDelayMinutes;

        @Field("is_cancelled")
        private Integer isCancelled;

        @Field("is_diverted")
```

```
        private Integer isDiverted;

        @Field("air_time_minutes")
        private Integer airTimeMinutes;
    }

    @Data
    @NoArgsConstructor
    @AllArgsConstructor
    public static class FlightLog {

        @Field("live_location")
        private GeoLocation location;
        private Integer altitude;
        private Integer speed;
        private Instant timestamp;
        private Instant eta;
    }
}
```

AirportMongo

```
public class AirportMongo {

    @Id
    private String id;
    private String name;
    private String city;
    private String state;
    private String country;
    private Location location;

    @Data
    @NoArgsConstructor
    public static class Location {
        private String type;
        private List<Double> coordinates;
    }
}
```

4.1.2 Neo4j Models

Route

```
public class Route {
```

```
    @RelationshipId
    @GeneratedValue
    private Long id;

    @Property("num_voli")
    private Integer numFlights;

    @Property("mean_scheduled_time")
    private Double meanScheduledTime;

    @TargetNode
    private Airport destination;
}
```

Airport

```
public class Airport {

    @Id
    @Property("iata_code")
    private String iataCode;

    @Property("name")
    private String name;

    @Property("latitude")
    private Double latitude;

    @Property("longitude")
    private Double longitude;

    // Relationships
    @Relationship(type = "LOCATED_IN", direction =
       Relationship.Direction.OUTGOING)
    private City city;

    @Relationship(type = "ROUTE", direction =
       Relationship.Direction.OUTGOING)
    private List<Route> routes = new ArrayList<>();
}
```

City

```
public class City {

    @Id
```

```
    @Property("city_state")
    private String cityState;

    @Property("name")
    private String name;

    @Property("state_id")
    private String stateId;


    @Property("population")
    private Integer population;

    @Property("density")
    private Double density;

    @Property("timezone")
    private String timezone;
}
```

4.2 Repository

The data access layer of our application leverages the abstraction provided by Spring Data. To manage the persistence of our domain entities, we developed five distinct repositories: User, Airport, AirportMongo, City, and Flight.

These components allow us to interface seamlessly with both MongoDB and Neo4j, depending on the data structure and the specific needs of the system. These repositories handle standard CRUD operations and basic user requests through derived queries. They also serve as the entry point for more sophisticated data processing. Specifically, we implemented complex aggregation pipelines and graph traversals for advanced analytics which we'll discuss farther on.

4.3 Service

The service layer acts as an intermediary between the controller and repository layers in the architecture of our application, representing its core functionalities and main business logic implementations.

Auth Service

This module manages user authentication and registration by verifying credentials and issuing *JWT tokens*. It ensures security through password encryption, as requested by non-functional requirements. The JWT token also contains information

on the role of the user, so we can gather that information directly instead of querying the database.

Custom User Details Service

The *CustomUserDetailsService* module combined with the *UserPrincipal* model integrates with Spring Security to load user details for authentication purposes. It retrieves users either by email MongoDB and wraps them in a UserPrincipal for use within the security context

User Service

This module implements all the requirements related to the user, serving user-related operations. It supports creating, retrieving and modifying users in the system.

Csv Flight Loader

This is an utility component dedicated to loading and parsing flight log data from the CSV file retrieved from the FlightRadar24 api into the system. It reads the file line by line, sanitizes the data, and maps the information into *FlightLogDTO* objects. These are then inserted into a time-ordered *PriorityQueue* to feed the air traffic simulation.

Flight Simulation Manager and Service

These two modules manage the system's simulated time flow, allowing for real-time air traffic visualization.

- The Manager implements *CommandLineRunner* and starts when the application boots. It uses the *CsvFlightLoader* to load logs into memory and triggers the simulation
- The Service operates asynchronously (@Async), consuming the priority queue of flight logs based on the advancement of the system clock (*SimulationClock*).

Flight Service

The FlightService acts as the core engine for all flight-related operations, handling the complex business logic across both the MongoDB and Neo4j databases. Beyond handling basic historical and real-time flight searches, the service manages advanced workflows such as cross-database routing for optimal itineraries and the aggregation of traffic statistics. Additionally, the service keeps Neo4j and MongoDB in sync whenever flight data changes. It also runs scheduled background jobs to track live flights and record their actual arrival delays

Airport Service

This is a central service that manages airport entities while maintaining consistency between the two NoSQL databases used: MongoDB (for geographical details) and Neo4j (for topological relationships and routing). It handles CRUD operations, ensuring they are transactional across both databases. It also provides advanced features for the Traffic Controller and the Airline Representative

Airline Service

This service manages airline statistics and reporting. It interacts with the *FlightRepository* and a simulated clock (*SimulationClock*) to filter data based on precise time intervals. It exposes functionalities to calculate the average delay, total flight volume, distances covered, and route-specific metrics.

City Service

Manages city-related operations and provides aggregate statistics for the Airline Representative. It calculates the most trafficked cities and generates hybrid statistics (total arrivals and departures) by aggregating data.

4.4 Controller

User Controller

Manages user accounts via secure REST endpoint (*/api/users*). It supports paginated user browsing, safe profile retrieval, and account deletion for the currently authenticated user. Crucially, it includes a protected endpoint (restricted to administrators via *@PreAuthorize*) for assigning or modifying user roles within the system.

Auth Controller

The authentication controller acts as the public entry point for system security (*/api/auth*). It exposes endpoints for user registration and authentication. Upon a successful login request, it validates the user's credentials and issues a JWT token, which is required to access protected routes within the application.

Flight Controller

The primary interface for flight and route management (*/api*). It exposes endpoints to search historical flights by date, track live active flights, and calculate the quickest real-world travel itineraries by combining graph topology with actual flight schedules. Furthermore, it delivers aggregate statistics on route delays, cancellations, and

diversions. It also handles flight CRUD operations, ensuring that creating, updating, or deleting a flight triggers the necessary graph relationship updates.

Airport Controller

Manages airport-related requests (*/api/airports*). It handles full CRUD operations that synchronously update both the Document DB and the Graph DB. It also exposes specialized analytical endpoints tailored for different system actors, such as emergency landing recommendations and alternative airport routing for the Traffic Controller , as well as top-hub analysis and connection rankings for the Airline Representative.

Airline Controller

Exposes REST endpoints (*/api/airlines*) for querying airline performance metrics. It provides route-specific and general rankings based on average delays, total flights, and distances covered. Additionally, it offers an endpoint to fetch comprehensive, aggregated reports for individual airlines over specified time intervals.

City Controller

Provides endpoints (*/api/cities*) to access data on traffic aggregations. It allows users to retrieve rankings of the most trafficked cities and detailed hybrid statistics (total arrivals and departures) for specific metropolitan cities. It also includes standard CRUD endpoints for managing city entities in the database.

4.5 File structure

The core codebase resides within the src/main/java directory and is logically separated into distinct functional layers: controller for REST API endpoints, service for business logic and transaction orchestration, and repository for database interactions. Data structures are neatly isolated into model and dto packages.

```
JavaProject/
├── data/
└── src/
    ├── main/
    │   ├── java/
    │   │   └── it/
    │   │       └── unipi/
    │   │           └── SkyGraph/
    │   │               ├── config/
    │   │               ├── controller/
    │   │               ├── dto/
    │   │               │   └── user/
    │   │               ├── enums/
    │   │               ├── exception/
    │   │               ├── model/
    │   │               ├── repository/
    │   │               └── service/
    │   └── resources/
    └── test/
        └── java/
            └── it/
                └── unipi/
                    └── SkyGraph/
```

Figure 4.1: Structure of SkyGraph Project

4.6 EndPoints

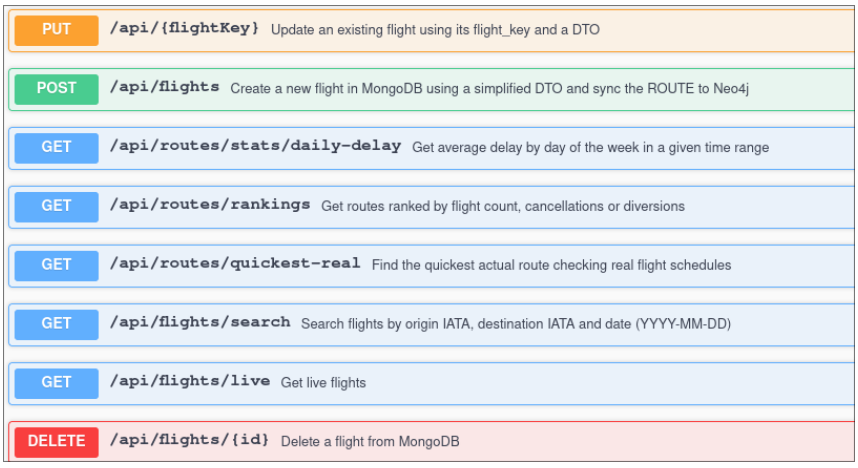

Figure 4.2: Flight Controller endpoints

The Flight Controller contains endpoints relative to the live and stored flights of the platform. It exposes CRUD operations for the flight collection, for instance, the creation of a new flight via POST /api/flights utilizes a simplified Data Transfer Object (DTO) to insert the primary record into MongoDB while simultaneously synchronizing the topological ROUTE relationships within the Neo4j graph database. Similar dual-database synchronization logic is supported during updates via PUT

/api/flightKey, which modifies an existing flight using its unique flight key and a DTO, and deletions via DELETE /api/flights/id.

The endpoint GET /api/flights/search allows users to query specific itineraries by filtering the extensive flight collection based on the origin IATA, destination IATA, and specific dates. Additionally, the GET /api/flights/live endpoint provides real-time flight tracking capabilities, returning all flights with a non-null flight log.

A significant portion of the API is devoted to analytical functions that use both document aggregation pipelines and graph traversal algorithms. The GET /api/routes/stats/daily-delay endpoint computes the average delay by day of the week within a specified time range. Furthermore, the GET /api/routes/rankings endpoint allows users to retrieve routes ranked by operational metrics such as total flight count, cancellations, or diversions. Finally, the GET /api/routes/quickest-real endpoint provides an heuristic method to find the quickest route between two airports using real flights stored in the database, thus using both the document database and the graph one.

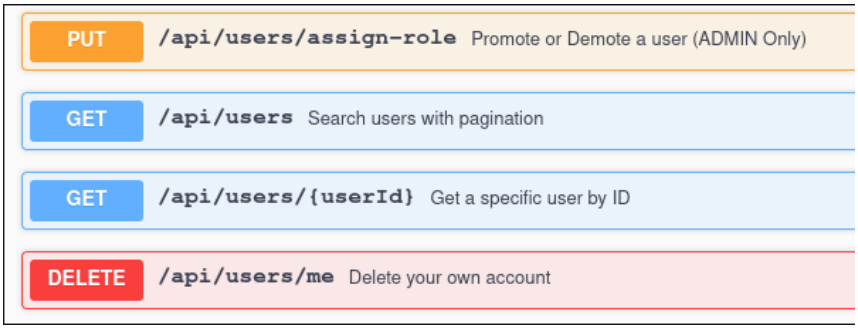

Figure 4.3: User Controller endpoints

The User Controller provides endpoints for identity management and profile retrieval within the system. It provides the GET /api/users endpoint, which allows for navigation through the user base with pagination support, and GET /api/users/userId for retrieving the detailed profile of a specific user by their unique identifier. The PUT /api/users/assign-role endpoint allows for the promotion or demotion of user privileges. Finally, the controller enables authenticated users to remove their own accounts via the DELETE /api/users/me endpoint.

Figure 4.4: Airport Controller endpoints

The Airport Controller manages the lifecycle and retrieval of airport data, providing straightforward CRUD capabilities alongside analytical queries. Data modification endpoints include POST /api/airports/, which creates a new airport and links it to a City via the LOCATED_IN relationship, and PUT /api/airports/iataCode, which synchronizes updates across both the Neo4j and MongoDB databases. Standard read operations retrieve either the complete list of airports via GET /api/airports or an individual record using GET /api/airports/iata, while DELETE /api/airports/iata handles entity removal.

The API supports graph-based network analysis and operational routing. The system ranks airports based on custom metrics and time intervals via GET /api/airports/rankings, or by total route volume using GET /api/airports/rankings/connections. Topological importance within the network is assessed by GET /api/airports/rankings/hubs, which returns top hubs based on centrality scores. Finally, the controller includes operational contingency endpoints: GET /api/airports/iata/alternative identifies the best alternative destinations in the event of an airport closure, and GET /api/airports/emergency determines the best emergency landing airports for a specific airborne flight.

Figure 4.5: City Controller endpoint

The City Controller manages the urban nodes within the aviation network, providing CRUD operations alongside aggregated traffic analytics. Standard data retrieval includes GET /api/cities for listing all city records and GET /api/cities/cityState for fetching a specific city by its unique composite identifier. Entity creation and updates are centralized through a single POST /api/cities endpoint, while removals are executed via DELETE /api/cities/cityState.

The GET /api/cities/city/stats endpoint calculates temporal operational metrics, returning the total number of flights, departures, and arrivals for a designated city within a specified time range. On a system-wide level, the GET /api/cities/rankings/traffic endpoint evaluates overall flight volume, allowing users to retrieve a ranked list of the most heavily trafficked cities in the network.

Figure 4.6: Auth Controller endpoints

The Auth Controller exposes two write operations to handle user registration and session initiation. The POST /api/auth/register endpoint processes new account creations, whilst the POST /api/auth/login endpoint authenticates existing users, validating their credentials to grant access to protected API resources by generating and returning a JWT token.

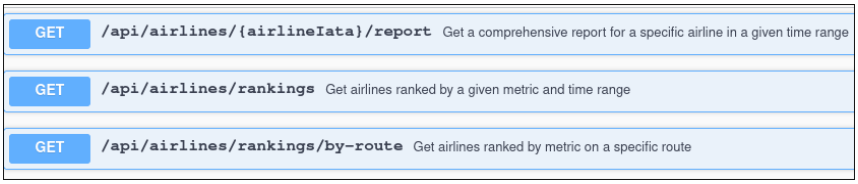

Figure 4.7: Airline Controller endpoint

The Airline Controller is dedicated to analytical operations, providing performance metrics and rankings for carriers operating within the network. To evaluate individual entities, the GET /api/airlines/airlineIata/report endpoint generates a report for a specific airline over a designated time range. The GET /api/airlines/rankings endpoint evaluates and ranks multiple airlines based on various metrics and timeframes. Furthermore, the GET /api/airlines/rankings/by-route endpoint allows users to assess and rank airline performance strictly across a specific flight route.

Chapter 5

Most Relevant Queries

5.1 MongoDB

findNearestAirport

This query is designed to support emergency landing scenarios handled by air traffic controllers. Given the real-time geographic coordinates of an aircraft in flight, the system retrieves the five nearest airports.

The implementation relies on MongoDB's `$geoNear` aggregation stage, which performs a geospatial proximity search using a `2dsphere` index defined on the `location` field of the `Airport` collection. The `spherical: true` option ensures that distances are computed using spherical geometry, providing accurate results over the Earth's surface. The `distanceMultiplier: 0.001` parameter converts the default distance (expressed in meters) into kilometers.

The pipeline then limits the result set to the five closest airports and projects only the relevant fields required by the `AirportEmergencyDTO`, namely the airport identifier, name, city, country, and computed distance in kilometers.

```
@Aggregation(pipeline = {  1 usage   Lorenzo Marranini
        """
        {
            $geoNear: {
                near: { type: 'Point', coordinates: [ ?0, ?1 ] },
                key: 'location',
                distanceField: 'distanceKm',
                spherical: true,
                distanceMultiplier: 0.001
            }
        }
        """,
        "{ $limit: 5 }",
        """
        {
            $project: { _id: 1, name: 1, city: 1, country: 1, distanceKm: 1 }
        }
        """
})
List<AirportEmergencyDTO> findNearestAirports(double longitude, double latitude);
```

Figure 5.1: Find nearest airport code

getAirlineReport

This MongoDB aggregation filters a specific airline's flights within a given timeframe and groups the main statistics as an analytics report. It calculates total flights, total and average distances, average delays, and an efficiency index based on the ratio between total kilometers and flight duration.

```
@Aggregation(pipeline = {  1 usage   Lorenzo Marranini
        "{ '$match': { " +
                "'flight_info.schedule.departure_datetime': { '$gte': ?0, '$lte': ?1 }, " +
                "'flight_info.airline.iata': ?2 " +
                "} }",
        "{ '$group': { " +
                "'_id': '$flight_info.airline.iata', " +
                "'totalKm': { '$sum': '$route.distance_km' }, " +
                "'avgKm': { '$avg': '$route.distance_km' }, " +
                "'totalFlights': { '$sum': 1 }, " +
                "'avgDelay': { '$avg': '$stats.tot_delay_minutes' }, " +
                "'totalDuration': { '$sum': '$stats.air_time_minutes' }" +
                "} }",
        "{ '$project': { " +
                "'_id': 0, " +
                "'airlineIata': '$_id', " +
                "'totalKm': 1, " +
                "'avgKm': 1, " +
                "'avgDelay': 1, " +
                "'totalFlights': 1, " +
                //calcolo efficiency
                "'efficiency': { " +
                "'$cond': [ " +
                "{ '$eq': ['$totalDuration', 0] }, " + // Se la durata è 0, evita divisione per zero
                "0, " +
                "{ '$divide': ['$totalKm', '$totalDuration'] }" +
                "] " +
                "}" +
                "} }"
})
List<AirlineReportDTO> generateAirlineReport(Instant start, Instant end, String AirlineIata);
```

Figure 5.2: Airline report code

getDelayByDayOfWeek

This pipeline filters flights within a given timeframe, groups them by day of the week, and calculates the average delay for each day, returning the days sorted from the highest average delay to the lowest.

```
@Aggregation(pipeline = {  1 usage  Lorenzo Marranini
        "{ '$match': { " +
                "'flight_info.schedule.departure_datetime': { '$gte': ?0, '$lte': ?1 }, " +
                "'stats.tot_delay_minutes': { '$ne': null } " +
                "} }",
        "{ '$project': { " +
                "'dayOfWeek': { '$dayOfWeek': '$flight_info.schedule.departure_datetime' }, " +
                "'delay': '$stats.tot_delay_minutes' " +
                "} }",
        "{ '$group': { " +
                "'_id': '$dayOfWeek', " +
                "'avgDelay': { '$avg': '$delay' } " +
                "} }",
        "{ '$sort': { 'avgDelay': -1 } }",
        "{ '$project': { " +
                "'_id': 0, " +
                "'dayOfWeek': '$_id', " +
                "'avgDelay': 1 " +
                "} }"
})
List<DayStatsDTO> findDaysByAvgDelay(Instant start, Instant end);
```

Figure 5.3: Avg delay per day of week code

5.2 Neo4j

getTopHubsByRank

This query for Neo4j identifies the best airport hubs by assigning an importance score. The score is calculated by combining direct flight traffic (weighted at 60%) and indirect flight traffic with one stop (weighted at 40%).

```
@Query("MATCH (airport:Airport)-[r1:ROUTE]->(dest:Airport) " +  1 usage  PaoloWalsh
        "WITH airport, sum(r1.num_voli) AS directTraffic " +

        "MATCH (airport)-[:ROUTE]->(dest:Airport)-[r2:ROUTE]->() " +
        "WITH airport, directTraffic, sum(r2.num_voli) AS indirectTraffic " +

        "RETURN airport.iata_code AS iata, " +
        "       airport.name AS name, " +
        "       (directTraffic * 0.6 + indirectTraffic * 0.4) AS score, '' AS scoreType " +
        "ORDER BY score DESC " +
        "LIMIT $limit")
List<AirportRankingDTO> findTopHubsByRank(@Param("limit") Integer limit);
```

Figure 5.4: Top hubs code

getBestAlternativeAirports

This query acts as a fallback when an airport is closed, searching for alternative hubs located within a 200 km radius. It calculates a suitability score based on the

number of shared destinations with the closed airport, penalizing the score based on geographic distance.

```
@Query("MATCH (closed:Airport {iata_code: $closedIata})-[:ROUTE]->(dest:Airport) " +  1 usage  LGranucci
        "MATCH (alt:Airport)-[:ROUTE]->(dest) " +
        "WHERE alt.iata_code <> $closedIata " +
        "WITH closed, alt, count(DISTINCT dest) AS sharedConnections " +

        "WITH alt, sharedConnections, " +
        "     point.distance( " +
        "       point({latitude: closed.latitude, longitude: closed.longitude}), " +
        "       point({latitude: alt.latitude, longitude: alt.longitude}) " +
        "     ) / 1000.0 AS distKm " +
        "WHERE distKm > 0 AND distKm < 200 " +

        "RETURN alt.iata_code AS iata, alt.name AS name, (sharedConnections / log(distKm + 1)) AS score, '' as scoreType " +
        "ORDER BY score DESC " +
        "LIMIT 5")
List<AirportRankingDTO> findBestAlternativeAirports(@Param("closedIata") String closedIata);
```

Figure 5.5: Best alternative airports code

5.3 Mongo + Neo4j

findQuickestRealRoute

These functions implement an advanced hybrid routing algorithm designed to find the fastest possible real travel itinerary. Given an origin airport, a destination, a specific departure date, and a maximum number of stops, it first uses Neo4j to fetch candidate routing paths. Then, it sequentially queries MongoDB's real future flight schedules to construct a valid journey. During this process, it strictly enforces a minimum 45-minute layover between any connecting flights to ensure the transfer is realistic. After filtering out any paths where consecutive flights aren't available, it evaluates the valid options and returns the complete itinerary with the shortest overall travel time.

```java
public TripItineraryDTO findQuickestRealRoute(String origin, String dest, String dateString, int maxHops) {  1 usage
    LocalDate date = LocalDate.parse(dateString, DateTimeFormatter.ISO_LOCAL_DATE);
    Instant tripStartTime = date.atStartOfDay(ZoneOffset.UTC).toInstant();
    List<String> rawCsvPaths = airportRepository.findCandidatePaths(origin, dest, maxHops);
    List<TripItineraryDTO> validItineraries = new ArrayList<>();
    for (String csvPath : rawCsvPaths) {
        List<String> path = Arrays.asList(csvPath.split( regex: ","));
        if (path.size() < 2) continue;
        List<FlightMongo> itineraryFlights = new ArrayList<>();
        Instant currentClock = tripStartTime;
        boolean validPath = true;
        //iterate over all candidate paths
        for (int i = 0; i < path.size() - 1; i++) {
            String legOrigin = path.get(i);
            String legDest = path.get(i+1);
            Instant minDeparture = (i == 0) ? currentClock : currentClock.plus(Duration.ofMinutes(45));
            //check for existance of suitable flight
            List<FlightMongo> flights = flightRepository.findNextFlight(
                    legOrigin,
                    legDest,
                    minDeparture,
                    PageRequest.of( pageNumber: 0,  pageSize: 1)
            );
            FlightMongo flight = flights.isEmpty() ? null : flights.get(0);
            if (flight == null) {
                validPath = false;
                break;
            }
            itineraryFlights.add(flight);
            currentClock = flight.getFlightInfo().getSchedule().getArrivalDatetime();
```

Figure 5.6: Real route code 1

```java
            currentClock = flight.getFlightInfo().getSchedule().getArrivalDatetime();
        }
        //check for path duration
        if (validPath && !itineraryFlights.isEmpty()) {
            Instant firstDep = itineraryFlights.get(0).getFlightInfo().getSchedule().getDepartureDatetime();
            Instant lastArr = itineraryFlights.get(itineraryFlights.size()-1).getFlightInfo().getSchedule().getArrivalDatetime();
            double totalMinutes = Duration.between(firstDep, lastArr).toMinutes();
            //add it to list
            validItineraries.add(new TripItineraryDTO(
                    totalMinutes,
                    stops: itineraryFlights.size() - 1,
                    itineraryFlights
            ));
        }
    }
    // return minimum value of validItineraries
    return validItineraries.stream()  Stream<TripItineraryDTO>
            .min(Comparator.comparingDouble(TripItineraryDTO::getTotalDurationMinutes))  Optional<TripItineraryDTO>
            .orElse( other: null);
}
```

Figure 5.7: Real route code 2

getCityHybridStats

This is a hybrid method that combines geographic information with operational data. It first retrieves all Iata codes associated with a given city that use the Neo4j DB and then queries MongoDB to count the actual number of total departures and

arrivals for those airports during a specific time range.

```java
public CityStatsDTO getCityHybridStats(String cityName, TimeInterval range) {  1 usage  PaoloWalsh
    Instant start = clock.calculateMinDate(range);
    Instant end = clock.getSimulatedNowInstant();

    List<String> iataCodes = cityRepository.findIataCodesByCity(cityName);
    String country = cityRepository.findCountryByCity(cityName);

    if (iataCodes == null || iataCodes.isEmpty()) {
        return new CityStatsDTO(cityName, country != null ? country : "Unknown",  TotalDepartures: 0L,  TotalArrivals: 0L,  TotalFlights: 0L,  AirportCount: 0);
    }

    long departures = flightRepository.countDeparturesByAirports(iataCodes, start, end);
    long arrivals   = flightRepository.countArrivalsByAirports(iataCodes, start, end);

    return new CityStatsDTO(cityName, country, departures, arrivals,  TotalFlights: departures + arrivals, iataCodes.size());
}
```

Figure 5.8: City hybrid stats code

5.4 Testing

To ensure the reliability of the SkyGraph system, a functional testing phase was conducted, aimed at verifying the correctness of the business logic and the consistency between the MongoDB and Neo4j databases. Tests were performed using API testing tools, specifically the integrated Swagger/OpenAPI dashboard. The methodology followed three main pillars:

- **Input Validation:** Both valid and invalid parameters were submitted to verify the server's error handling and response accuracy.
- **Visual Inspection:** Results returned by the controllers were cross-referenced with the database states using MongoDB shell and Neo4j cypher.
- **Role-Based Access Control:** Verification was conducted to ensure that protected endpoints (e.g., `PUT /api/users/assign-role`) were accessible exclusively to users with the `ADMIN` role, while unauthorized requests were correctly rejected.

This testing methodology was conducted on all implemented endpoints to validate their correctness and integration. The following section presents a subset of representative test cases.

- The creation of a new flight was verified via the `POST /api/flights` endpoint. Manual inspection confirmed the correct insertion of the document into MongoDB and the simultaneous creation of the corresponding `ROUTE` relationship in Neo4j.
- Using the `GET /api/routes /quickest-real` endpoint, the calculation of itineraries with multiple hops was tested. It was visually verified that the results strictly adhered to the minimum 45-minute layover requirement imposed by the algorithm.

- An airport failure was simulated using the `GET /api/airports/{iata}/alternative` endpoint. The system correctly returned alternative hubs within a 200 km radius, ranked by their similarity score.

5.5 Database Indexing and Optimization

To ensure horizontal scalability, low-latency responses, and optimal query execution across both databases, indexes were defined. The objective was to transform linear-time operations ($O(n)$) into logarithmic or constant-time lookups ($O(\log n)$ or $O(1)$), minimizing unnecessary full scans and reducing computational overhead.

MongoDB Indexing Strategy

Within the MongoDB database, three primary indexes were implemented to support the application's queries.

First, an index on `flight_info.schedule.departure_datetime` was created to optimize temporal range queries on the `flights` collection. Since most queries filter flights by date, the absence of this index would result in a full collection scan across more than 4.7 million documents.

Second, a unique index on `flight_info.flight_key` was defined to guarantee data integrity and prevent duplicate flight entries. This index also enables near-constant-time retrieval for individual flight lookups.

Third, a `2dsphere` index was configured on the `location` field of the `airports` collection to enable efficient geospatial proximity searches. Thanks to this index MongoDB can drastically reduce the search space when computing distance-based queries.

The performance impact of these indexes was evaluated using the `explain("executionStats")` framework. The most significant improvement was observed in the temporal range query on the `flights` collection, as shown in Table 5.1.

Test Scenario	Docs Examined	Docs Returned	Time (ms)
Without Index	4,772,506	18,175	49,344
With Index	18,175	18,175	618

Table 5.1: Temporal Range Query on `flights` Collection

The index reduced execution time from approximately 49 seconds to 618 milliseconds. More importantly, the number of examined documents dropped from the entire collection size to exactly the number of returned documents, achieving a perfect selectivity ratio of 1:1.

Additional validation was performed for unique key lookups and geospatial proximity queries, as summarized in Table 5.2.

The unique key lookup exhibits near $O(1)$ behavior, while the geospatial index demonstrates high efficiency by examining only five documents to retrieve four

Test Scenario	Docs Examined	Docs Returned	Time (ms)
flight_key	1	1	25
Geospatial Near 50km	5	4	144

Table 5.2: Execution Stats for Unique and Geospatial Lookups

matching airports, effectively pruning the search space by several orders of magnitude.

Neo4j Indexing Strategy

In parallel, the Neo4j graph database was optimized to support route computation and entity resolution. A uniqueness constraint was defined on the `iata_code` property of `Airport` nodes, effectively serving as a primary key. This guarantees entity uniqueness and accelerates point lookups during route expansion queries.

Additionally, a range index was created on the `name` property of `City` nodes to enable efficient searches and improve geographic entity mapping.

Execution	DB Accesses	Total Time (ms)
With Index	8	473
Without Index	630	512

Table 5.3: Data obtained from execution of "Get city by name" query

Execution	DB Accesses	Total Time (ms)
With Index	6	239
Without Index	690	298

Table 5.4: Data obtained from execution of "Get airport by iata_code" query

Overall, the combined indexing strategies across MongoDB and Neo4j ensure that both document-based and graph-based workloads operate with predictable performance characteristics, even under large-scale datasets.

Chapter 6

AI Tools Usage

In accordance with the project requirements, this section outlines the utilization of Large Language Models (LLMs), specifically Gemini and ChatGPT, during the development of the project.

6.1 Purpose and Tasks Performed

AI tools were integrated into the workflow to accelerate development and refine the technical documentation. The primary tasks included:

- **Boilerplate Code Generation:** Creating standard Spring Boot structures, such as Data Transfer Objects (DTOs), Repository interfaces, and basic Controller mappings to reduce repetitive manual coding.
- **Debugging and Configuration:** Assisting in identifying misconfigurations within the `application.properties` file and resolving dependency conflicts in the `pom.xml`.
- **Brainstorming:** Evaluating different ideas for the initial project scope and suggesting different dataset sources.
- **System Deployment:** Providing guidance for deploying the MongoDB Replica Set and the Spring Boot application on the assigned virtual machines via SSH.
- **Documentation and Language Refinement:** Improving the formal tone and grammatical accuracy of the final report.

6.1.1 Critical Evaluation and Modifications

The integration of AI tools provided a significant acceleration in the initial development phases, particularly for generating boilerplate structures and streamlining repetitive coding tasks. However, the AI's suggestions were often treated as preliminary drafts rather than final solutions.

While these tools were helpful in speeding up the workflow, we frequently encountered code errors and overly generalized advice that did not account for the specific complexities of our architecture. To overcome these limitations, we relied on both class notes and the official documentation for Spring Boot, MongoDB, and Neo4j.

Ultimately, while AI served as a valuable assistant, the technical integrity of SkyGraph was ensured through manual debugging and direct reference to authoritative technical sources.